Project Proposal — ECE 379K

Team name: HST Team
Members (name + EID): Tamim Hasan (th34492), Humza Ahmad (hsa695), Samuel Li (sl55355)
Date: 09/21/2026

1. What we are building

We are building a pipeline that turns raw telescope images into a catalog of detected stars, galaxies, and other bright objects — automatically, and fast enough to handle thousands of images at once.

Today, sifting through raw astronomical images to find scientifically interesting objects is slow and largely manual, especially at the scale of modern sky surveys that produce thousands of exposures a night. Our project builds a pipeline that reads standard telescope images (FITS format), removes noise and imaging artifacts, and automatically finds and measures bright object in each image — its location, size, and brightness — without a person needing to look at it. We split the work between CPU and GPU: CPU handles reading files and organizing the work, GPU handles the heavy numerical filtering, so the pipeline can eventually process large batches efficiently rather than one image at a time. By the end of the semester, we will validate the system on synthetic test images containing objects with known position, size, and brightness, and measure how closely the pipeline recovers those values. We will consider the performance goal met if the final system processes the same image batches at least 2x as fast as a version that uses only the CPU, without reducing measurement accuracy.

2. Who is doing what

Team member	Owns
Tamim Hasan (th34492)	Builds and maintains image ingestion and output: reads FITS files, writes the final detection catalog
Humza Ahmad (hsa695)	Builds region-tiling logic and the parallel-processing infrastructure that lets multiple images/tiles run at once
Samuel Li (sl55355)	Builds the noise-filtering/enhancement stage and the source-detection algorithm that measures each object

What we will do together: We will agree on the interface between components, debug the first end-to-end integration, and write the final report.

Dependencies, and what the waiting person does meanwhile: The three workstreams can begin in parallel: Tamim builds ingestion, Humza tests tiling and concurrency on representative image data, and Samuel tests processing and detection on synthetic images. Once ingestion produces real images, the pieces will be integrated incrementally while each owner continues testing and refining their component.

Milestones:

By when	What exists	Who
Oct 15	Single-threaded baseline: one FITS image in, detection catalog out	Tamim: ingestion Humza: tiling on synthetic arrays Samuel: detection on synthetic images

By when	What exists	Who
Oct 31	Real images tiled and processed concurrently with 2–4 workers; detection validated on synthetic sources	Tamim + Humza integrate ingestion/tiling Samuel integrates detection into Humza's pipeline
Nov 15	GPU filtering/enhancement works and matches the CPU reference numerically	Samuel leads Tamim supports validation
Dec 1	Full ingest → tile → filter → detect → output pipeline runs on image batches; validation and limitations documented	All three

3. What could go wrong

What could go wrong	How we would notice early	What we would do instead
GPU filtering takes too long to implement or fails validation	By Nov 15 the GPU path is not running or does not match the CPU reference	Keep filtering on CPU with SciPy FFT
Threading or tiling causes races or stalls	Oct 16–31 tests hang or produce inconsistent results	Use one shared 2–4 worker pool that processes whole images
A team member becomes unavailable	A check-in or milestone deliverable is missed	Reassign remaining work and cut optional features first
TACC GPU access is unreliable	Queue waits exceed a day or GPU jobs repeatedly fail	Use smaller batches, fewer nodes, or CPU nodes
Tiling creates boundary artifacts	Detection accuracy drops near tile boundaries	Skip tiling and process whole images

Reduced scope: If necessary, we will deliver a minimally parallel CPU pipeline that reads FITS images, performs background estimation and source detection without tiling, and outputs source ID, centroid, bounding box, size, and integrated brightness. We will validate it on synthetic images with known injected sources.

4. What we will need

What	Status	Needed by
TACC Lonestar6 GPU resources (approximately one GPU node for development and benchmarking, 20-30 hours)	Have access, can get allocation	Nov 1
TACC CPU nodes for baseline and parallel testing	Have	Oct 1
Python with astropy, NumPy, SciPy	Have (via TACC modules / pip)	Oct 1
CUDA toolkit + CuPy	Can get (TACC module)	Nov 1
500 calibrated r-band FITS images from the Sloan Digital Sky Survey (6 GB uncompressed)	Can download from the public SDSS archive	Oct 15
Synthetic test FITS images with known injected sources	Need to generate ourselves	Oct 15

Nothing is needed from the instructor at this time.